UNIDADES TECNOLÓGICAS DE SANTANDER

Tecnología en Desarrollo de Sistemas Informáticos

TecnoTurno

Informe inicial del proyecto de gestión de reparaciones

Aplicación web con Spring Boot para organizar la recepción, el diagnóstico y el seguimiento de equipos en un taller tecnológico.

Integrantes

Manuel Santiago Terán Suárez
msteran@uts.edu.co

Harvic Edisson Quintero Cáceres
hedissonquintero@uts.edu.co

Docente
Carlos Adolfo Beltrán Castro

Bucaramanga
31 de octubre de 2026

1 Planteamiento y justificación

Problema que aborda el proyecto

En un taller que lleva sus reparaciones mediante notas o archivos separados, resulta difícil relacionar al propietario con su equipo, conocer el técnico responsable y reconstruir los cambios realizados. La falta de un registro común puede ocasionar consultas repetidas, pérdida de información y confusión sobre el estado o el costo de una reparación. El proyecto se orienta a talleres tecnológicos que requieren un registro centralizado de sus servicios.

Solución propuesta

Proponemos TecnoTurno como una aplicación web que centraliza clientes, técnicos, equipos y órdenes de servicio. Cada orden conserva la falla reportada, el diagnóstico, el costo y una secuencia de eventos con fecha y responsable. El administrador organiza la recepción; el técnico registra los avances; el cliente consulta únicamente sus reparaciones.

Justificación

El proyecto busca mejorar la organización de la información y facilitar el seguimiento de los equipos recibidos. Un historial común permite explicar qué se hizo y quién registró cada avance, mientras que los permisos por perfil limitan el acceso a los datos de cada cliente. La disponibilidad de búsquedas y filtros facilita localizar reparaciones sin revisar registros dispersos.

Desde el componente académico, la propuesta permite aplicar desarrollo web con Java, persistencia relacional, validación de formularios, separación de responsabilidades y construcción de interfaces con Thymeleaf y Bootstrap. Su alcance permite presentar un proceso completo de negocio y relacionarlo con los requerimientos, el modelo de datos y las pruebas.

Alcance de la primera versión

Incluye administración de usuarios y equipos, recepción y asignación inicial de órdenes, diagnóstico, costo, estados, consulta por perfiles, historial y panel de seguimiento. No incluye pagos, inventario de repuestos, facturación electrónica, notificaciones externas, recuperación de contraseña por correo ni reasignación posterior de técnicos.

2 Objetivos y entorno de trabajo

Objetivo general

Desarrollar una aplicación web con Spring Boot que permita gestionar la recepción y el seguimiento de reparaciones tecnológicas, centralizando la información de usuarios, equipos y órdenes de servicio mediante perfiles de acceso y un historial de actualizaciones.

Objetivos específicos

- Identificar los datos y las reglas necesarios para registrar usuarios, equipos y órdenes de servicio.
- Diseñar una base de datos relacional con restricciones que conserven la integridad y trazabilidad de las reparaciones.
- Implementar formularios validados para administrar usuarios y equipos, recibir órdenes y registrar avances técnicos.
- Establecer permisos de administrador, técnico y cliente para controlar las consultas y modificaciones.
- Construir una interfaz adaptable con plantillas reutilizables y mensajes que comuniquen el resultado de las operaciones.
- Verificar mediante pruebas de integración el acceso por perfiles, las validaciones y el ciclo de una reparación.

Entorno propuesto

Componente	Selección
IDE	Spring Tools con soporte de proyectos Maven
Lenguaje y compilación	Java 17 y Maven 3.9.9 usado en la verificación
Aplicación	Spring Boot 3.5.7 y Tomcat embebido
Persistencia	Spring Data JPA e Hibernate; MySQL 8.0.16+
Demostración	H2 en archivo con perfil demo
Interfaz	Thymeleaf y Bootstrap 5.3.3 mediante WebJars
Validación y acceso	Jakarta Bean Validation y Spring Security con BCrypt
Control de versiones	Git y repositorio público en GitHub

3 Perfiles y reglas de operación

Perfil	Responsabilidades
Administrador	Crear y editar usuarios; gestionar equipos; registrar órdenes y asignar técnic actualizar cualquier orden.
Técnico	Consultar y actualizar exclusivamente las órdenes que tiene asignadas; regis costo, estado y nota.
Cliente	Consultar sus órdenes, el diagnóstico, el costo y el historial; buscar y filtrar s registros.

Reglas de negocio

RN01 El administrador registra clientes, técnicos, equipos y órdenes; el técnico actualiza las órdenes asignadas; el cliente solo consulta las propias.

RN02 Los perfiles no cambian después del registro. Una cuenta puede desactivarse; el administrador no puede desactivar su propia cuenta desde el formulario.

RN03 Correo, documento y serial no se repiten. El correo se almacena en minúsculas y el serial en mayúsculas.

RN04 El costo corresponde al valor registrado de la reparación en pesos colombianos; no representa una factura electrónica ni procesa pagos.

RN05 El avance de estado es secuencial. Se admite conservar el estado para agregar una nota o corregir diagnóstico y costo mientras la orden esté abierta.

RN06 La cancelación está disponible desde RECIBIDO, EN_DIAGNOSTICO y EN_REPARACION. ENTREGADO y CANCELADO son estados terminales.

RN07 Las fechas de las reparaciones se registran en la zona horaria de Bogotá. Cada cambio conserva su responsable.

RN08 No se eliminan órdenes, historiales ni usuarios desde la aplicación. Los equipos se eliminan únicamente cuando no tienen órdenes.

RN09 La recepción bloquea la fila del equipo dentro de la transacción antes de comprobar si existe una orden abierta, para evitar altas simultáneas duplicadas.

Estados de una reparación

RECIBIDO → EN_DIAGNOSTICO → EN_REPARACION → LISTO → ENTREGADO. La cancelación es posible antes de LISTO. Mientras la orden esté abierta se pueden añadir notas sin cambiar el estado.

4 Requerimientos funcionales parte 1

RF01 Autenticación

Ingresar con correo y contraseña; rechazar credenciales erróneas y cuentas inactivas; cerrar sesión.

Criterio de aceptación: Login correcto redirige al panel; incorrecto muestra aviso.

RF02 Perfiles

Separar permisos de administrador, técnico y cliente tanto en interfaz como en servidor.

Criterio de aceptación: Un cliente recibe HTTP 403 al intentar administrar usuarios.

RF03 Usuarios

Crear y editar cuentas con nombre, documento, correo, teléfono, perfil y contraseña; activar o desactivar.

Criterio de aceptación: Correo y documento son únicos; el perfil se conserva después del alta.

RF04 Equipos

Registrar y editar tipo, marca, modelo, serial, accesorios y cliente propietario.

Criterio de aceptación: Se exige cliente activo y serial único normalizado en mayúsculas.

RF05 Eliminación controlada

Eliminar equipos solo si no tienen órdenes vinculadas.

Criterio de aceptación: Un equipo con historial se conserva y se muestra una advertencia.

RF06 Recepción

Crear una orden con equipo, falla y técnico responsable; asignar estado RECIBIDO y fecha.

Criterio de aceptación: Cada recepción crea simultáneamente su primer registro de historial.

4 Requerimientos funcionales parte 2

RF07 Asignación

Asignar a cada nueva orden un técnico activo.

Criterio de aceptación: No se acepta una cuenta de cliente como técnico.

RF08 Órdenes simultáneas

Evitar registrar una nueva orden para un equipo que ya tiene una abierta.

Criterio de aceptación: Se permite una nueva recepción tras ENTREGADO o CANCELADO.

RF09 Seguimiento

Consultar listado y detalle de órdenes con cliente, equipo, técnico, falla, diagnóstico, costo y fechas.

Criterio de aceptación: El cliente ve sus órdenes y el técnico únicamente las asignadas.

RF10 Búsqueda y filtros

Buscar por identificador OT, serial o nombre del cliente y filtrar por estado.

Criterio de aceptación: Las búsquedas se aplican después de restringir por usuario.

RF11 Actualización

Registrar diagnóstico, costo no negativo, estado y nota de avance.

Criterio de aceptación: Diagnóstico y nota son obligatorios; costo admite hasta dos decimales.

4 Requerimientos funcionales parte 3

RF12 Flujo de estados

Aplicar RECIBIDO → EN_DIAGNOSTICO → EN_REPARACION → LISTO → ENTREGADO; cancelar antes de LISTO.

Criterio de aceptación: No se admite saltar de RECIBIDO a ENTREGADO ni modificar órdenes cerradas.

RF13 Trazabilidad

Guardar estado, nota, fecha y responsable de cada actualización en la misma transacción.

Criterio de aceptación: Cada avance aparece en el historial cronológico; no hay edición del historial.

RF14 Panel

Mostrar total de órdenes visibles, órdenes en proceso, listas para entrega y cinco órdenes recientes.

Criterio de aceptación: Los indicadores respetan el perfil autenticado.

RF15 Mensajes

Mostrar confirmaciones, advertencias y errores de validación.

Criterio de aceptación: Guardar usuario o equipo muestra éxito; eliminación bloqueada muestra advertencia.

RF16 Protección de vínculos

Conservar propietario de un equipo con órdenes y no eliminar usuarios ni órdenes.

Criterio de aceptación: No se puede transferir un historial a un nuevo propietario mediante edición del equipo.

5 Requerimientos no funcionales parte 1

RNF01 Seguridad de acceso

Proteger todas las rutas de negocio y comprobar propiedad o asignación antes de leer o modificar una orden.

Criterio de aceptación: Pruebas de administrador, cliente y técnico sin asignación.

RNF02 Credenciales

Guardar contraseñas con BCrypt; mínimo ocho caracteres en altas; nunca mostrar el hash.

Criterio de aceptación: Inspección de registros y pruebas de autenticación.

RNF03 Protección web

Mantener CSRF activo, usar POST para modificaciones y escape de salida con Thymeleaf.

Criterio de aceptación: POST sin token rechazado con HTTP 403; datos se muestran con th:text.

RNF04 Integridad

Usar claves foráneas, valores obligatorios, restricciones únicas y transacciones para orden e historial.

Criterio de aceptación: Scripts SQL y pruebas de duplicados e historial.

RNF05 Concurrencia

Detectar actualizaciones simultáneas sobre una orden mediante versión optimista.

Criterio de aceptación: Un formulario con versión anterior se rechaza sin sobrescribir cambios.

RNF06 Usabilidad

Mostrar etiquetas visibles, errores junto al campo, confirmación al eliminar y navegación común.

Criterio de aceptación: Revisión de formularios, mensajes y flujo de trabajo.

5 Requerimientos no funcionales parte 2

RNF07 Adaptación visual

Permitir uso desde móvil y escritorio con Bootstrap y tablas desplazables.

Criterio de aceptación: Revisión a 390 px y 1366 px; no debe existir desbordamiento general.

RNF08 Mantenibilidad

Separar modelo, repositorios, servicio, controladores, configuración y plantillas reutilizables.

Criterio de aceptación: Revisión de paquetes, fragmentos head/header/footer y código fuente.

RNF09 Portabilidad

Ejecutar como aplicación Maven con Java 17 y Tomcat embebido; perfiles demo H2 y MySQL.

Criterio de aceptación: Compilar y ejecutar el perfil demo; validar esquema MySQL antes de uso.

RNF10 Rendimiento

Objetivo: responder en menos de dos segundos en consultas habituales con hasta 1000 órdenes en equipo de desarrollo.

Criterio de aceptación: Medir en el entorno de despliegue; objetivo no certificado por las pruebas actuales.

RNF11 Recuperación

Conservar datos de demo en archivo y permitir respaldo MySQL con mysqldump.

Criterio de aceptación: Reinicio de demo y procedimiento documentado de respaldo/restauración.

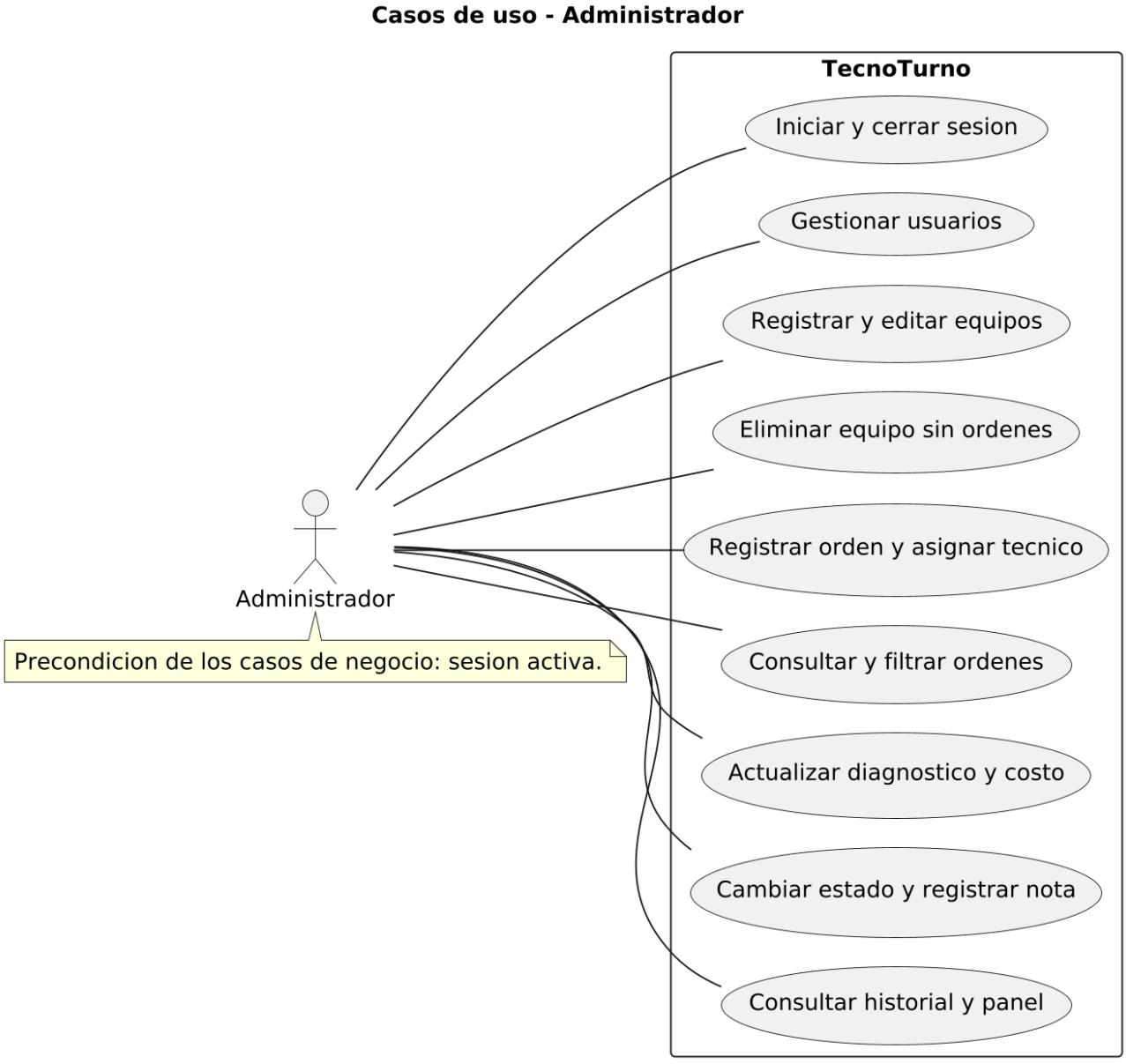
RNF12 Sesión y errores

Caducar sesión a los 30 minutos de inactividad y no exponer trazas internas en páginas de error.

Criterio de aceptación: Configuración de sesión y manejo de errores.

6 Casos de uso del administrador

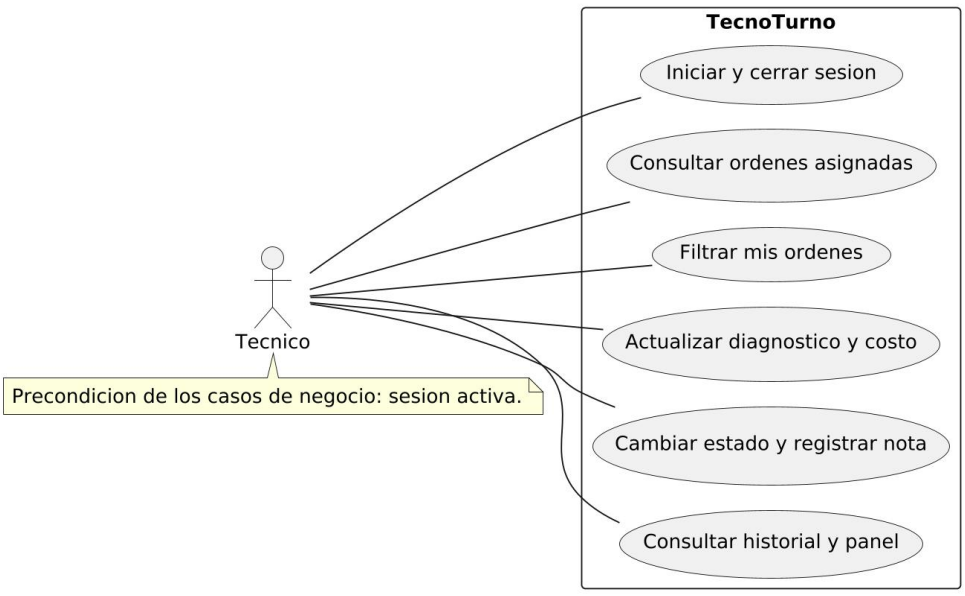
Los casos de negocio requieren autenticación previa. El administrador dispone de acceso a todas las órdenes; las restricciones de eliminación protegen los historiales existentes.



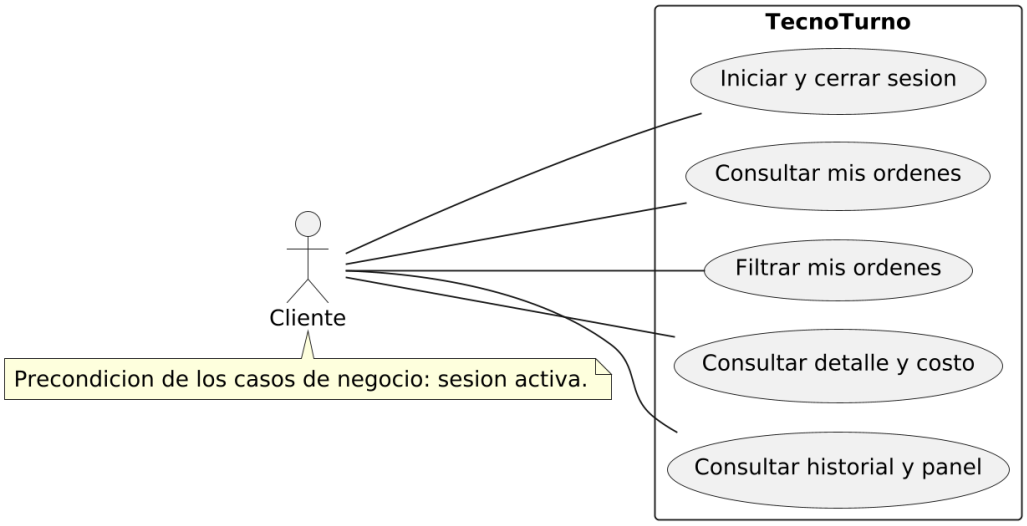
7 Casos de uso del técnico y del cliente

El acceso a una orden se comprueba en el servidor: el técnico debe estar asignado y el cliente debe ser propietario del equipo.

Casos de uso - Tecnico



Casos de uso - Cliente



8 Especificación de los casos principales

CU01 Registrar una reparación

Actor: administrador. Precondiciones: sesión activa, cliente y técnico activos, equipo registrado sin orden abierta. Flujo: seleccionar equipo y técnico, describir falla y guardar. El sistema valida los datos, bloquea la fila del equipo, comprueba órdenes abiertas y crea la orden RECIBIDO junto con su primer evento. Resultado: orden e historial persistidos; confirmación de éxito. Alternativa: si faltan datos o hay una orden abierta, se muestran errores y no se crea la recepción.

CU02 Actualizar una reparación

Actores: técnico asignado o administrador. Precondición: orden abierta y visible. Flujo: abrir detalle, indicar diagnóstico, costo, estado y nota, y guardar. El sistema valida permisos, campos, transición y versión; guarda orden e historial en una transacción. Alternativas: transición inválida, costo negativo o formulario desactualizado impiden guardar.

CU03 Consultar seguimiento

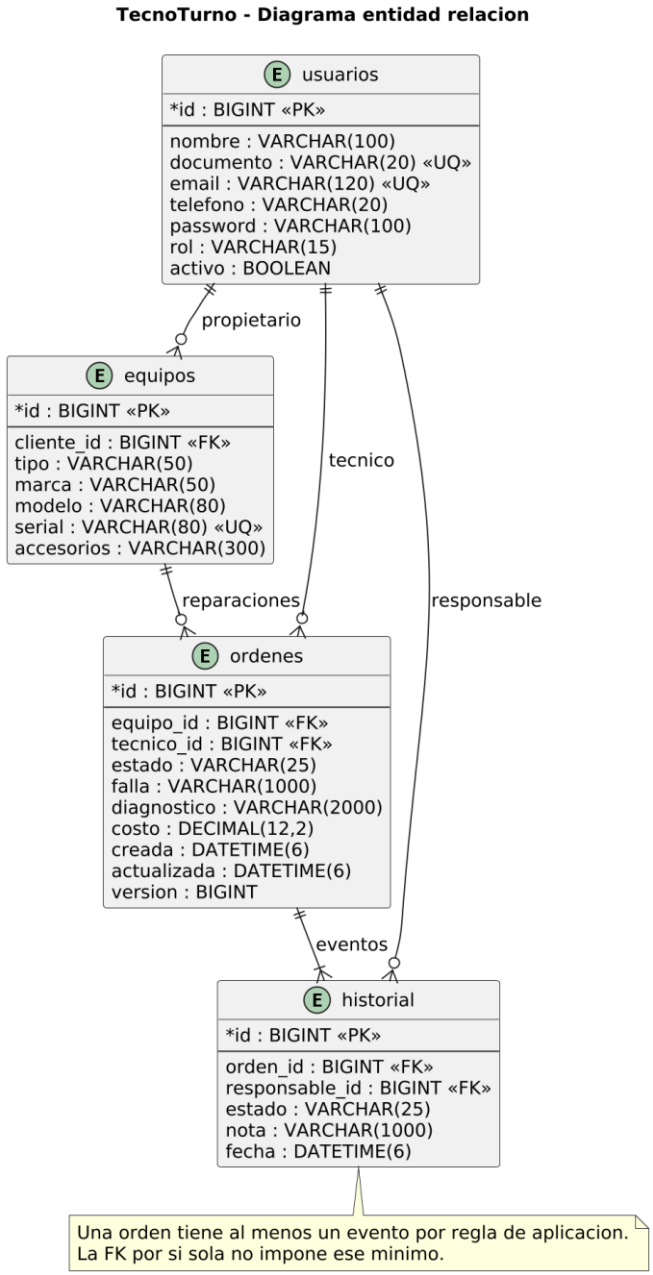
Actores: los tres perfiles. Flujo: ingresar, buscar una orden por número, cliente o serial, aplicar filtro y abrir el detalle. Resultado: se muestran datos e historial de una orden autorizada. Alternativas: no hay coincidencias o el usuario no tiene permiso; se informa el resultado sin exponer órdenes ajenas.

CU04 Eliminar un equipo

Actor: administrador. Flujo: elegir un equipo y confirmar eliminación. Si no tiene órdenes, el sistema elimina y muestra aviso. Si existen órdenes, conserva el equipo y advierte la restricción. No hay eliminación de historial.

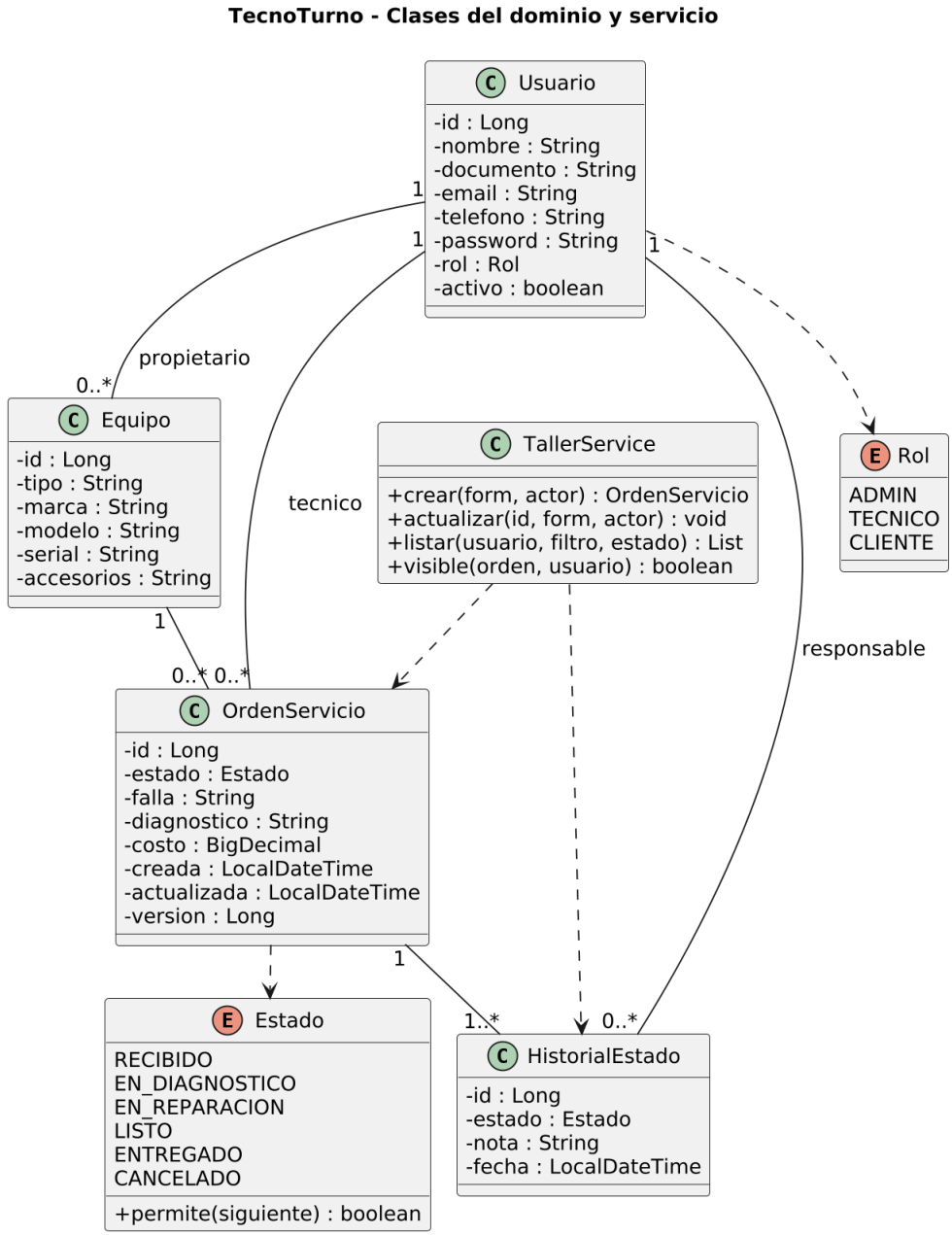
9 Diagrama entidad relación

El modelo contiene cuatro tablas. Usuarios integra las cuentas de los tres perfiles; los vínculos de propietario y técnico se validan según el rol. Las claves foráneas conservan las relaciones y no utilizan borrado en cascada.



10 Diagrama de clases

Las entidades representan la información persistente. TallerService concentra autorización por registro, recepción transaccional, avance de estados y trazabilidad. Los controladores reciben las solicitudes de los usuarios y los repositorios gestionan el acceso a los datos.



11 Base de datos y diccionario

Tabla	Clave y vínculos	Datos principales
usuarios	PK id; UQ documento y email	nombre, teléfono, hash password, rol y activo
equipos	PK id; FK cliente_id → usuarios; UQ serial	tipo, marca, modelo y accesorios
ordenes	PK id; FK equipo_id → equipos y tecnico_id → usuarios	estado, falla, diagnóstico, costo DECIMAL(12,2), fechas y versión
historial	PK id; FK orden_id → ordenes y responsable_id → usuarios	estado, nota y fecha del evento

Normalización y restricciones

Los datos de una persona no se repiten en cada reparación: la orden referencia al equipo y al técnico, y el equipo referencia a su propietario. Los eventos de seguimiento se almacenan en filas independientes. Cada tabla tiene una clave primaria y no contiene listas de valores en una misma celda. Los textos de diagnóstico y nota describen eventos distintos, no sustituyen las relaciones.

Los campos obligatorios tienen NOT NULL, los identificadores de negocio tienen UNIQUE y las referencias cuentan con FOREIGN KEY. El script MySQL incluye CHECK para roles, estados y costo no negativo. La coherencia entre rol y función del usuario, el flujo de estados y la existencia del primer evento se garantizan en la aplicación.

Scripts entregados

- 01_crear_base_datos.sql crea la base tecnoturno y sus cuatro tablas sin borrar datos existentes.
- 02_insertar_datos.sql agrega tres cuentas ficticias, dos equipos, dos órdenes y tres eventos; las contraseñas están cifradas con BCrypt.
- 03_consultas_verificacion.sql permite consultar relaciones e identificar órdenes abiertas duplicadas.

Los scripts se ejecutan por orden y una sola vez en una base nueva. El perfil mysql valida el esquema; el perfil demo utiliza H2 y crea sus propios ejemplos al primer inicio. Los registros de ejemplo permiten comprobar las relaciones entre las tablas y el seguimiento de una reparación.

12 Referencias

Beltrán Castro, C. A. (s. f.). Ajustes funcionales y de presentación en Spring Boot. Unidades Tecnológicas de Santander.

Spring. (s. f.). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/3.5/

Spring. (s. f.). Validating Form Input. https://spring.io/guides/gs/validating-form-input/

Thymeleaf. (s. f.). Documentation. https://www.thymeleaf.org/documentation.html

Bootstrap. (s. f.). Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/